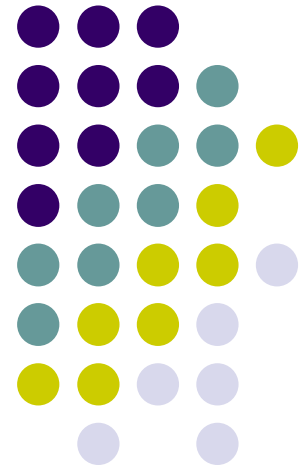
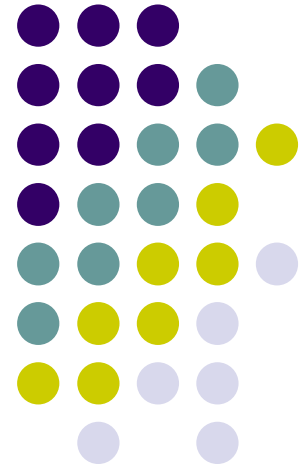


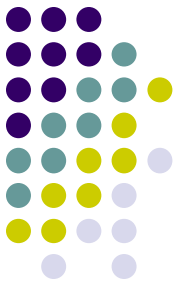
UNIT II

Basic Processing Unit



Some Fundamental Concepts

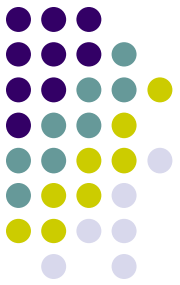




Fundamental Concepts

- Processor fetches one instruction at a time and perform the operation specified.
- Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered.
- Processor keeps track of the address of the memory location containing the next instruction to be fetched using Program Counter (PC).
- Instruction Register (IR)-Instruction fetched from the memory are kept in IR in CPU.

Executing of an Instruction



❖ Fetch Phase

- Fetch the contents of the memory location pointed to by the PC. The contents of this location are loaded into the IR (fetch phase).

$$IR \leftarrow [[PC]]$$

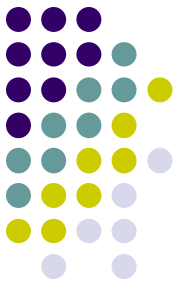
- Assuming that the memory is byte addressable, increment the contents of the PC by 4 (fetch phase).

$$PC \leftarrow [PC] + 4$$

❖ Execution Phase

- Carry out the actions specified by the instruction in the IR (execution phase).

SINGLE BUS ORGANIZATION OF PROCESSOR



In a single bus organisation, all major units of the processor (registers, ALU, memory interface, etc.) are connected to a common internal bus.

Only one data transfer can occur at a time across this bus.

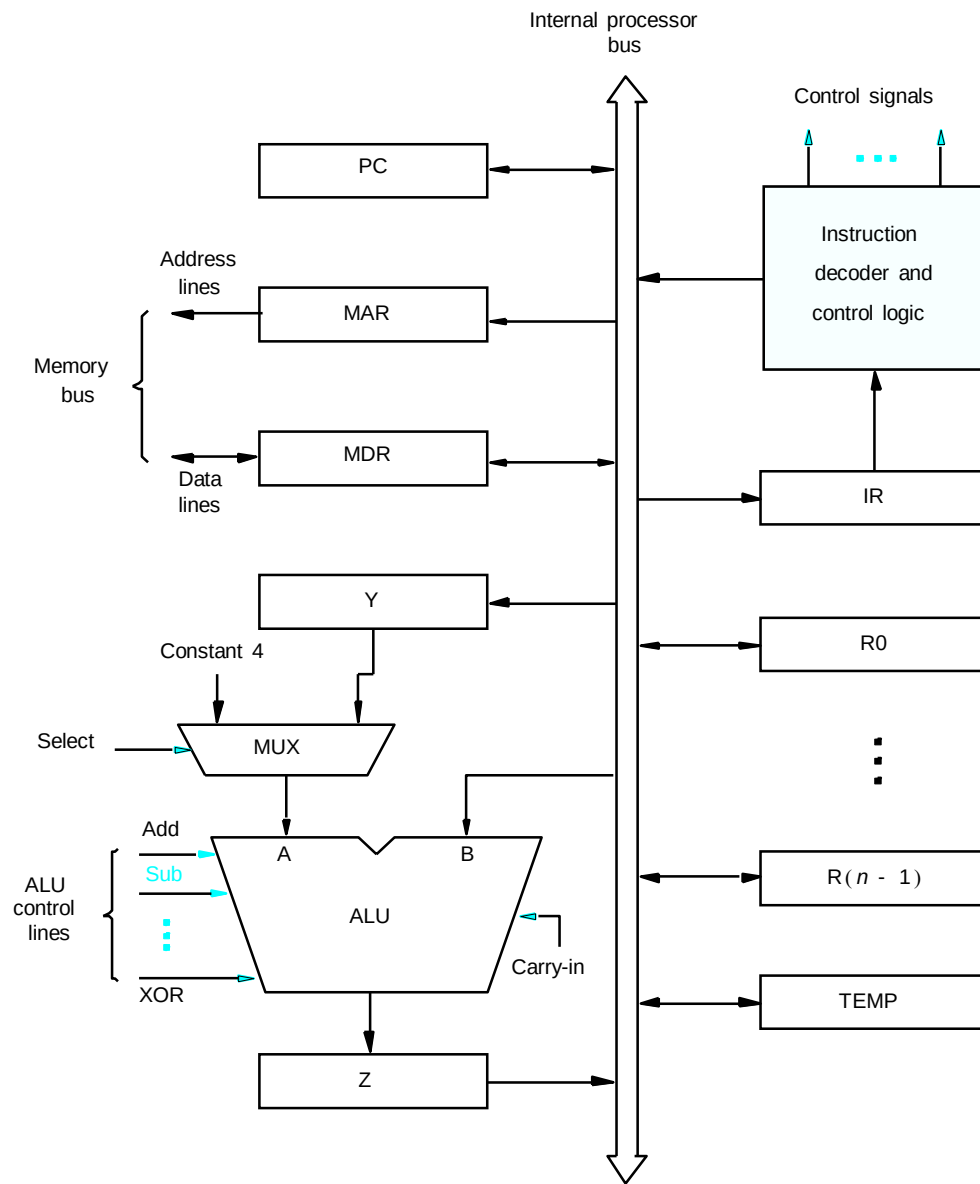
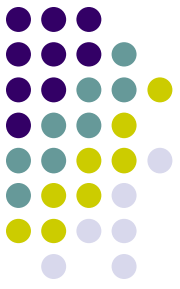


Figure 7.1. Single-bus organization of the datapath inside a processor.

MAIN COMPONENTS

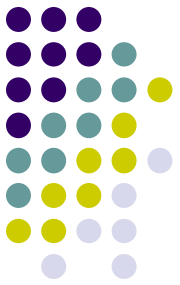


Single Internal Processor Bus:

- Connects all units.
- Only one unit can send data at a time; others must wait.

Registers:

- PC (Program Counter): Points to the next instruction.
- IR (Instruction Register): Stores current instruction.
- RO to R(n-1): General-purpose registers.
- TEMP, Y, Z: Temporary storage for operations.



Memory Interface:

- MAR (Memory Address Register): Holds address for memory access.
- MDR (Memory Data Register): Holds data from/to memory.

Control Unit:

- Instruction Decoder and Control Logic: Generates control signals for operations.

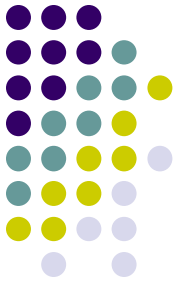
ALU (Arithmetic Logic Unit):

- Performs arithmetic and logic operations using inputs A and B.
- Gets input from Y and MUX.
- Outputs result to Z.

MUX (Multiplexer):

- Selects between constant values or register data as ALU input.

HOW IT WORKS ?



Fetch Phase:

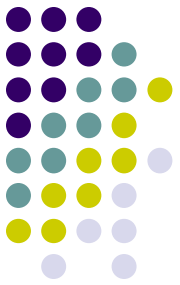
- PC $\rightarrow$ MAR $\rightarrow$ Memory $\rightarrow$ MDR $\rightarrow$ IR.
- Instruction is fetched using a sequence of bus transfers.

Decode Phase:

- IR sends opcode to the control unit.
- Control signals are generated.

Execute Phase:

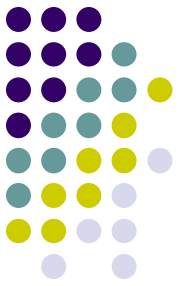
- Operands moved via bus to ALU.
- ALU performs operation.
- Result stored in Z or another



Executing an Instruction

- Transfer a word of data from one processor register to another or to the ALU.
- Perform an arithmetic or a logic operation and store the result in a processor register.
- Fetch the contents of a given memory location and load them into a processor register.
- Store a word of data from a processor register into a given memory location.

Register Transfers



- All operations and data transfers are controlled by the processor clock.

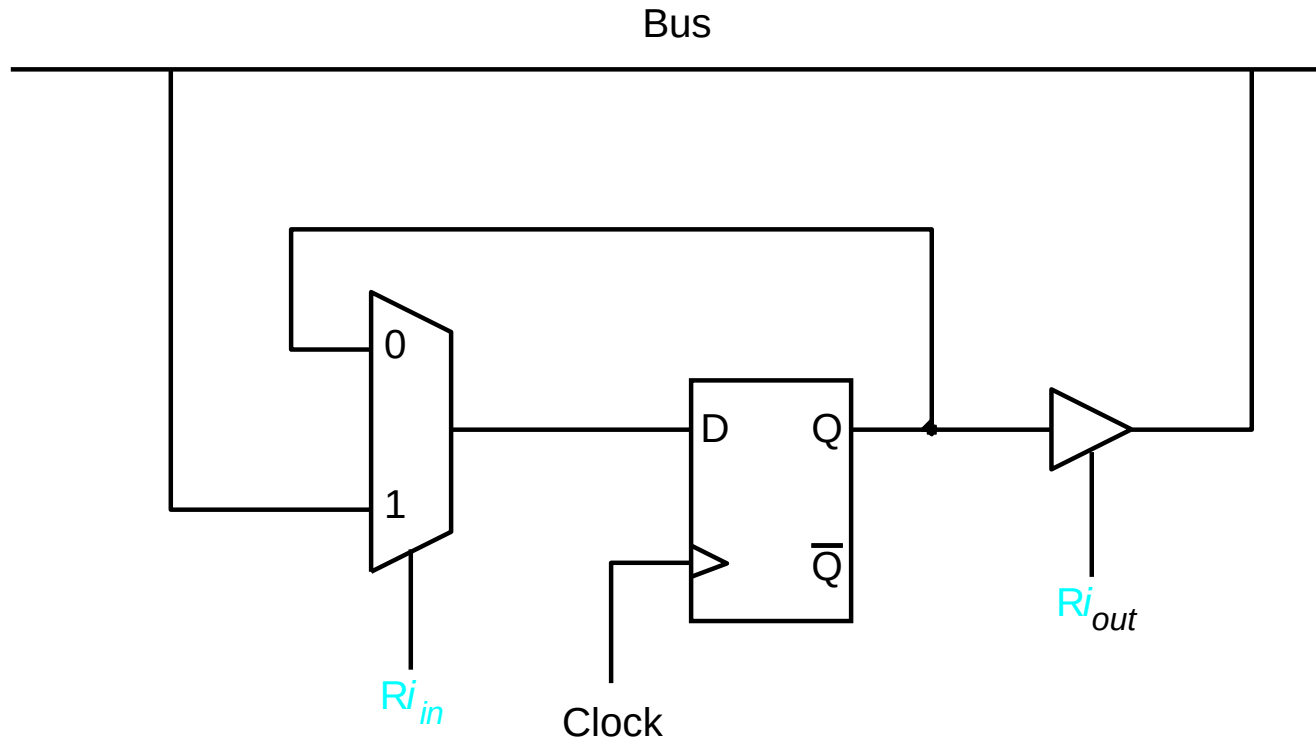
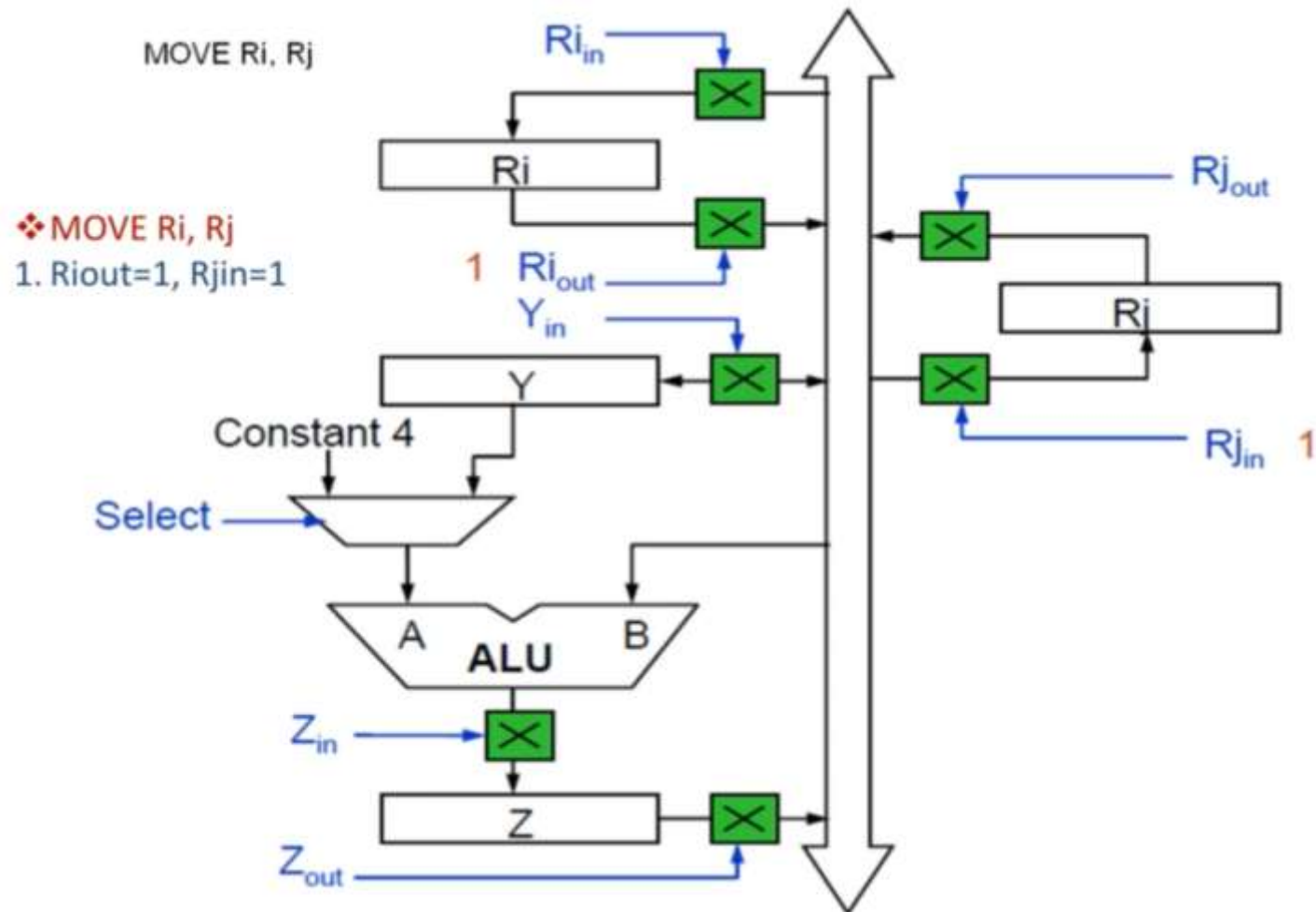
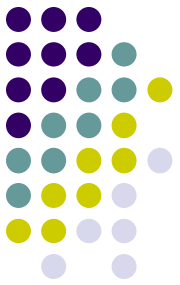
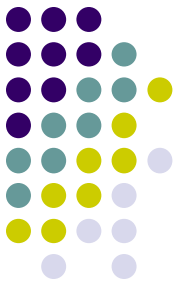


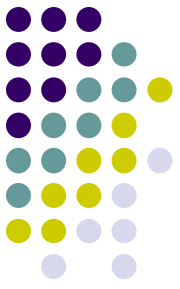
Figure 7.3. Input and output gating for one register bit.

Register Transfers





- Key Elements and Flow Explanation:
- 1. Ri (General-Purpose Register):
 - Stores a value that can be used as an operand.
 - Rin: Control signal to load data into Ri from the bus.
 - Rout: Control signal to place data from Ri onto the bus.
- 2. Y Register:
 - Temporary register to hold the first operand for the ALU.
 - Yin: Loads data from the bus into Y.
- 3. MUX (Multiplexer):
 - Selects between a constant value or data from another source for the second operand to the ALU.



- 4. ALU (Arithmetic Logic Unit):
- Performs operations like Add, Sub, AND, OR, etc.
- Takes two inputs:
- A from MUX
- B from Y
- Produces result based on ALU control inputs.
- 5. Z Register:
- Temporary register to store ALU output.
- Zin: Loads result from ALU.
- Zout: Places result from Z onto the internal bus.

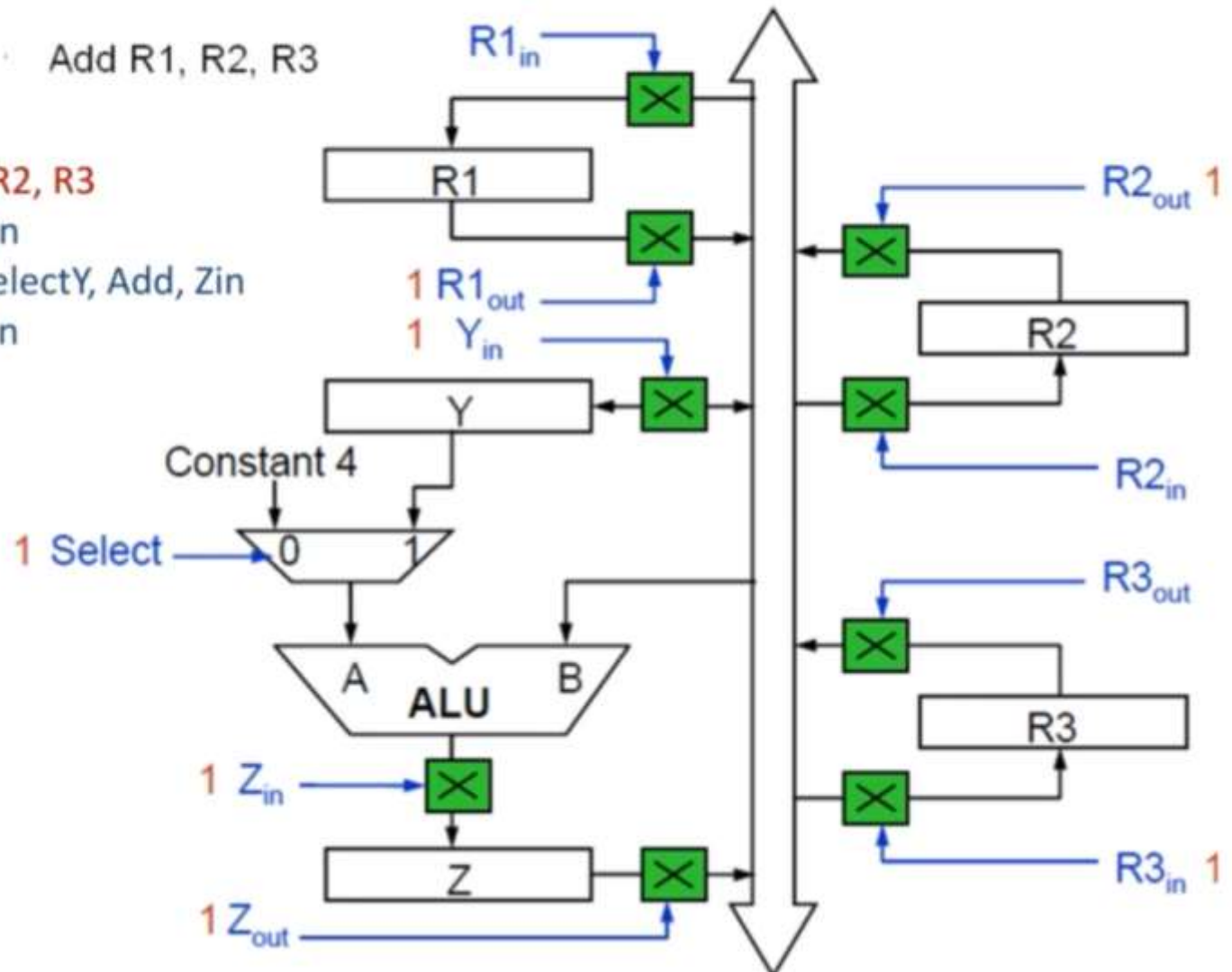


Arithmetic Operation

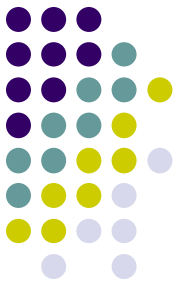
Add R1, R2, R3

❖ Add R1, R2, R3

1. R1out, Yin
2. R2out, SelectY, Add, Zin
3. Zout, R3in

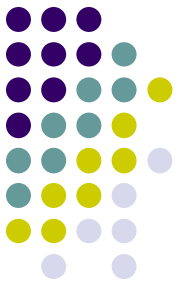


Performing an Arithmetic or Logic Operation



- The ALU is a combinational circuit that has no internal storage.
- ALU gets the two operands from MUX and bus. The result is temporarily stored in register Z.
- What is the sequence of operations to add the contents of register R1 to those of R2 and store the result in R3?
 1. R1out, Yin
 2. R2out, SelectY, Add, Zin
 3. Zout, R3in

Fetching a Word from Memory



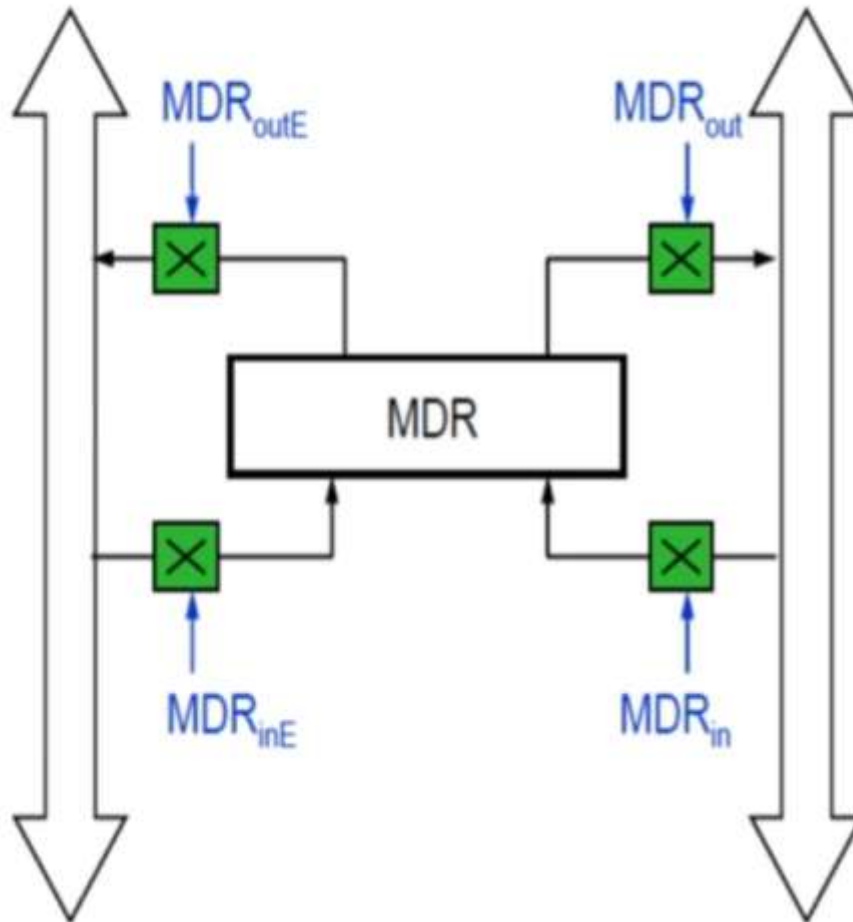
- The response time of each memory access varies (cache miss, memory-mapped I/O,...).
- To accommodate this, the processor waits until it receives an indication that the requested operation has been completed (Memory-Function-Completed, MFC).
- Move (R1), R2
 - $MAR \leftarrow [R1]$
 - Start a Read operation on the memory bus
 - Wait for the MFC response from the memory
 - Load MDR from the memory bus
 - $R2 \leftarrow [MDR]$

Fetching a Word from Memory

- Address into MAR; issue Read operation; data into MDR.

Memory data bus

Internal processor bus

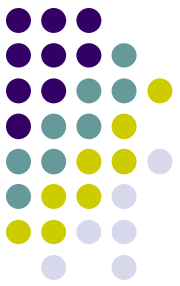


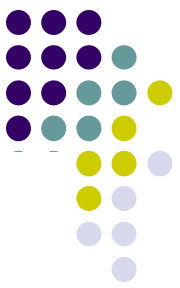
Move (R1), R2

1. $R1_{out}$, MAR_{in} , Read
2. MDR_{inE} , WMFC
3. MDR_{out} , $R2_{in}$

WMFC: is the control signal that causes the processor's control circuitry to wait for the arrival of the MFC signal.

Connection and control signals for register MDR.





4. Storing a word in memory

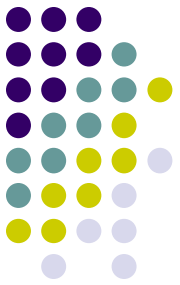
- Address is loaded into MAR
- Data to be written loaded into MDR.
- Write command is issued.
- Example:
- **Move R2, (R1)**

$R1_{out}, MAR_{in}$

$R2_{out}, MDR_{in}, Write$

$MDR_{outE}, WMFC, End$

Execution of a Complete Instruction



- Add (R3), R1
- Fetch the instruction
- Fetch the first operand (the contents of the memory location pointed to by R3)
- Perform the addition
- Load the result into R1

Architecture

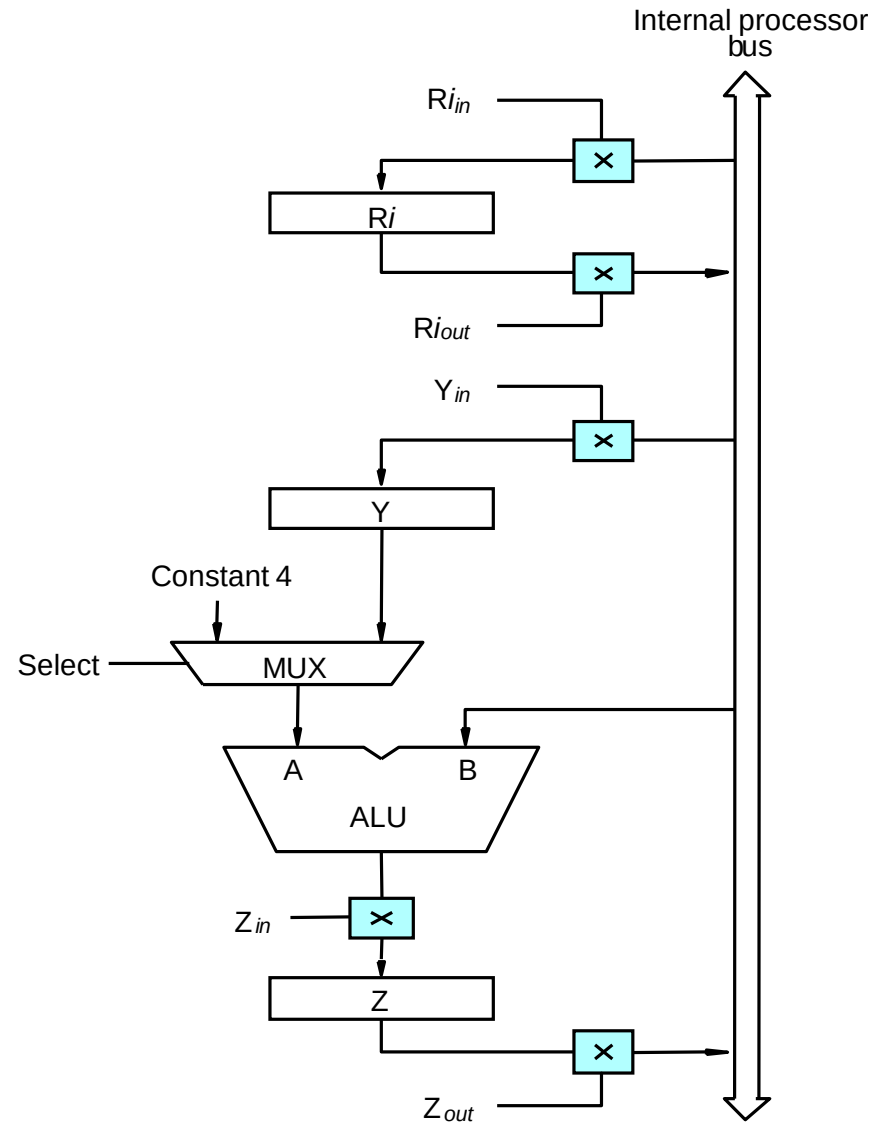
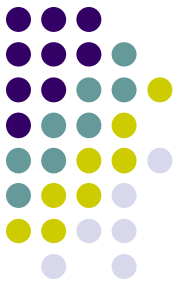


Figure: Input and output gating for the registers.

Execution of a Complete Instruction

Add (R3), R1

Step	Action
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMF C
3	MDR_{out} , IR_{in}
4	$R3_{out}$, MAR_{in} , Read
5	$R1_{out}$, Y_{in} , WMF C
6	MDR_{out} , SelectY, Add, Z_{in}
7	Z_{out} , $R1_{in}$, End

Figure: Control sequence for execution of the instruction Add (R3),R1.

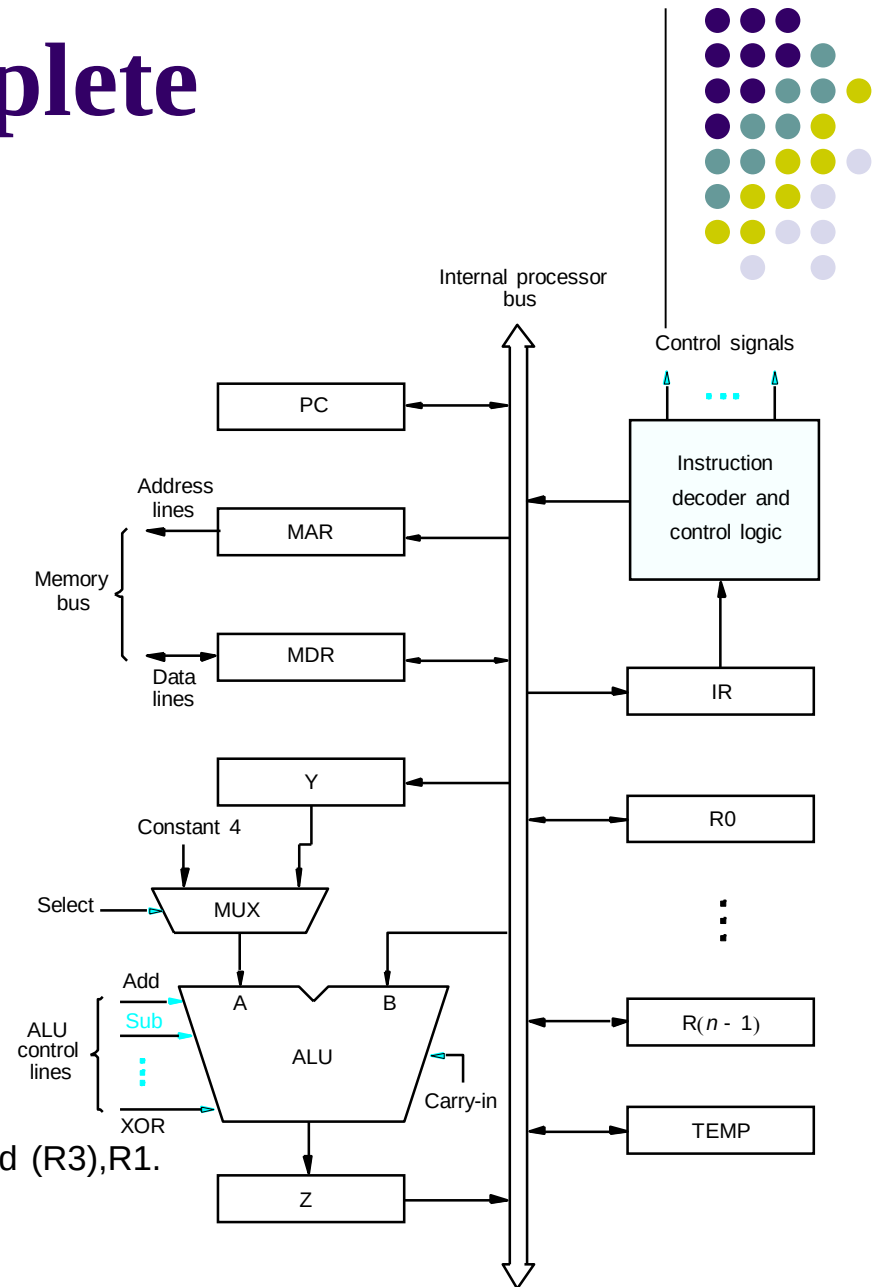
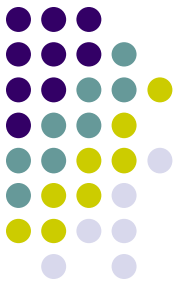
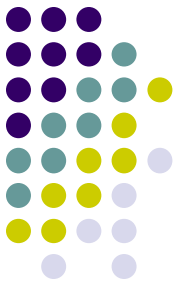


Figure 7.1. Single-bus organization of the datapath inside a processor.

Execution of Branch Instructions



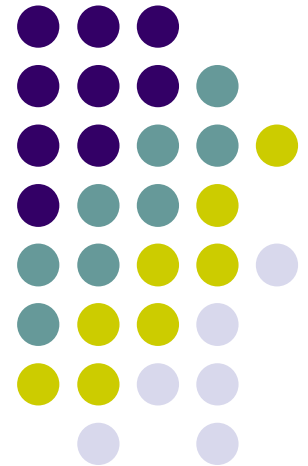
- A branch instruction replaces the contents of PC with the branch target address, which is usually obtained by adding an offset X given in the branch instruction.
- The offset X is usually the difference between the branch target address and the address immediately following the branch instruction.
- Conditional branch

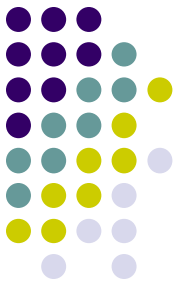


Step	Action
1	PC_{out} , MAR_{in} , Read, Select4, Add, Z_{in}
2	Z_{out} , PC_{in} , Y_{in} , WMF C
3	MDR_{out} , IR_{in}
4	Offset-field-of- IR_{out} , Add, Z_{in}
5	Z_{out} , PC_{in} , End

Figure : Control sequence for an unconditional branch instruction.

Pipelining

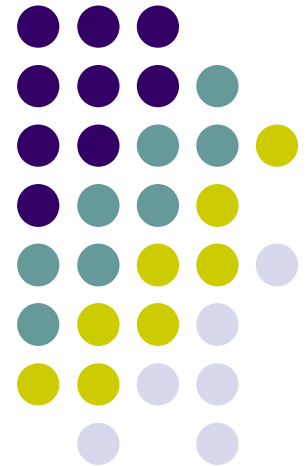




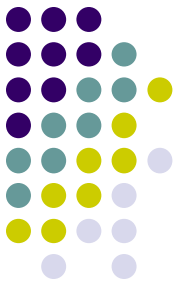
Overview

- Pipelining is widely used in modern processors.
- Pipelining improves system performance in terms of throughput.
- Pipelined organization requires sophisticated compilation techniques.

Basic Concepts

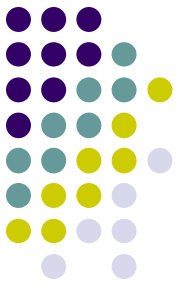


Making the Execution of Programs Faster

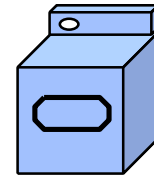
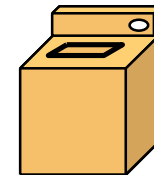
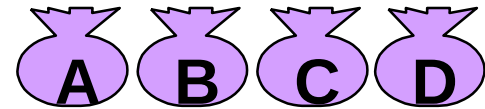


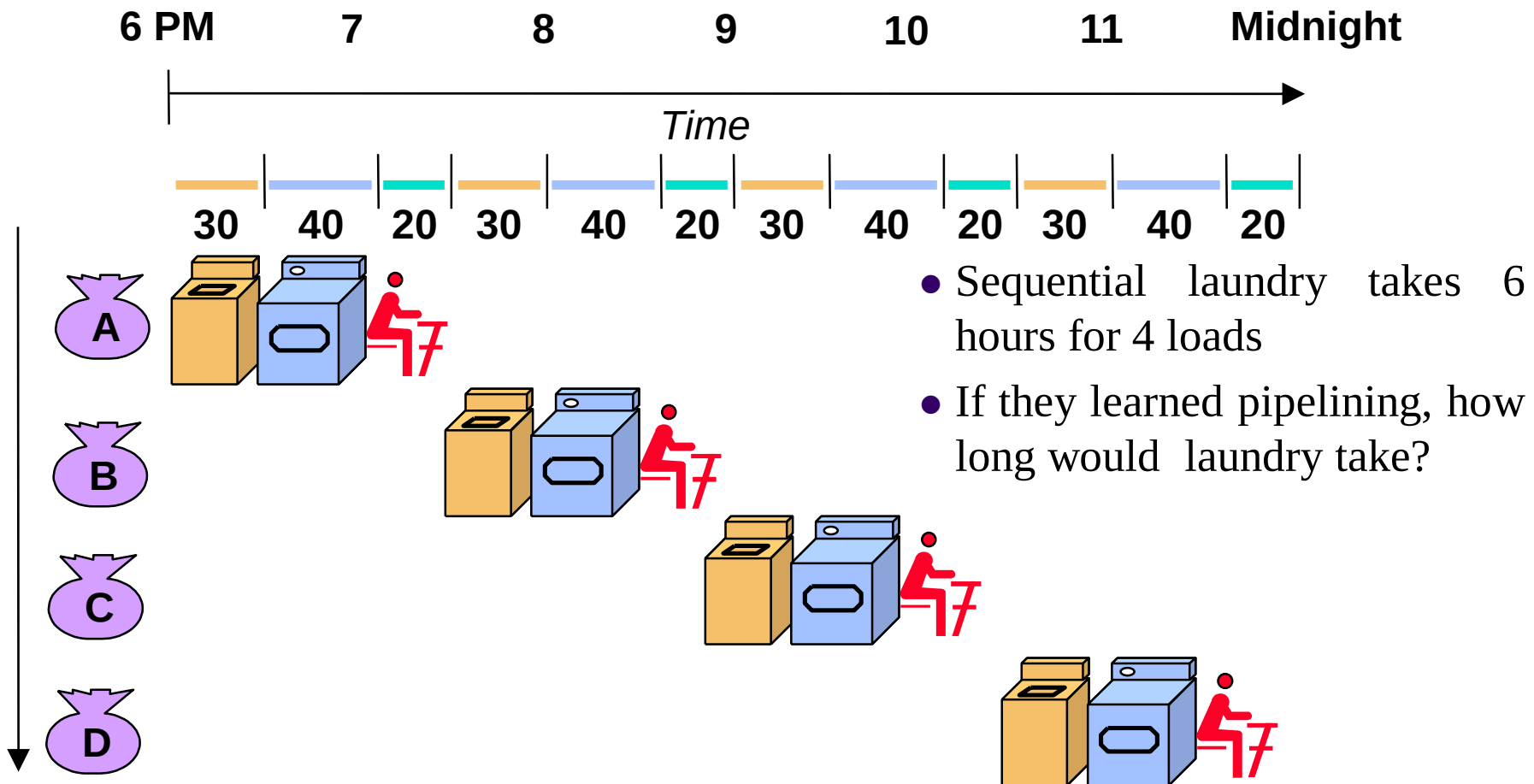
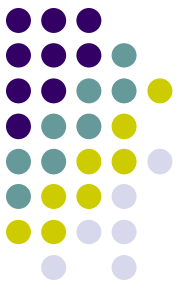
- Use faster circuit technology to build the processor and the main memory.
- Arrange the hardware so that more than one operation can be performed at the same time.
- In the latter way, the number of operations performed per second is increased even though the elapsed time needed to perform any one operation is not changed.

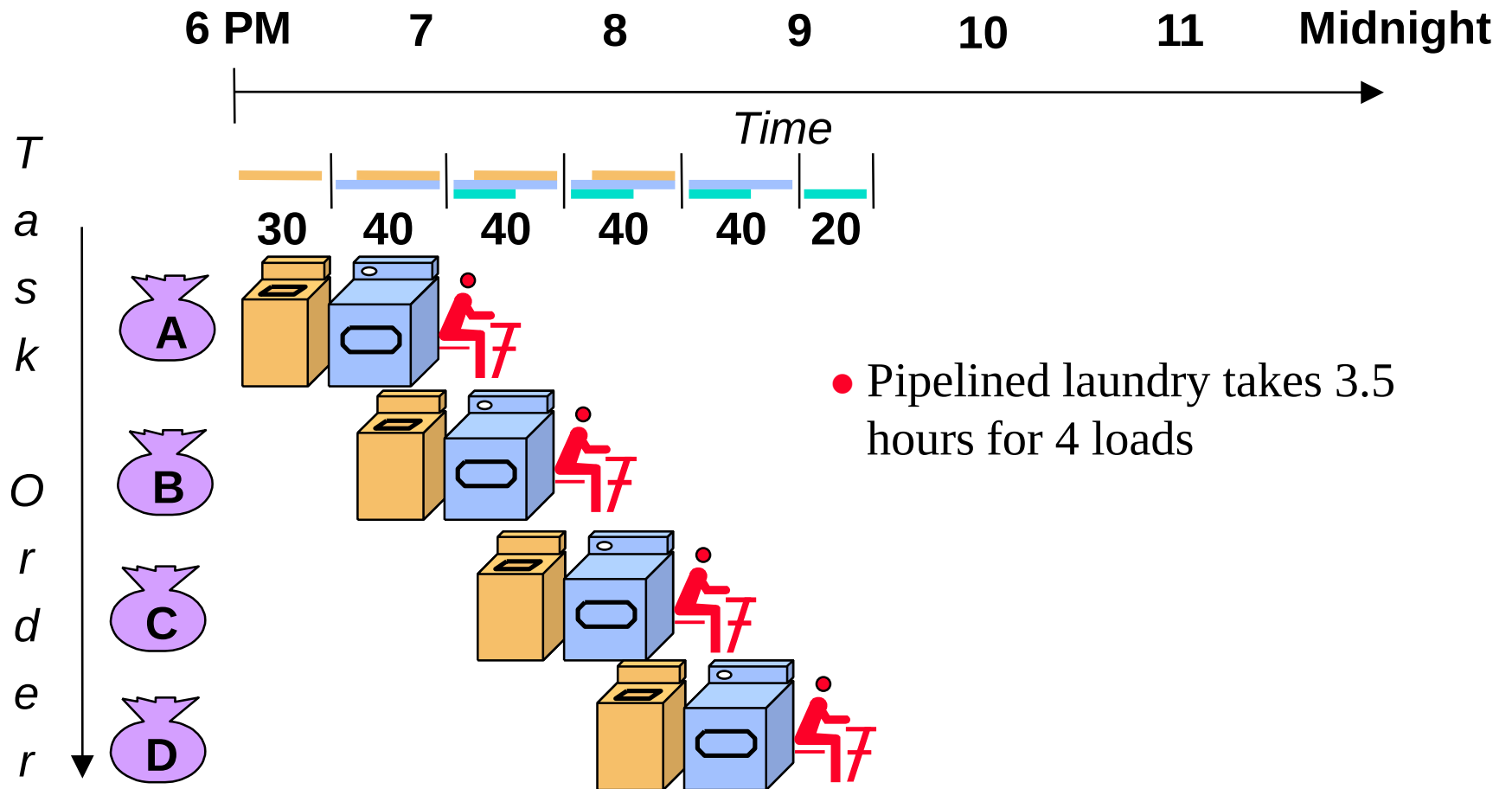
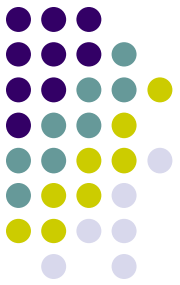
Traditional Pipeline Concept

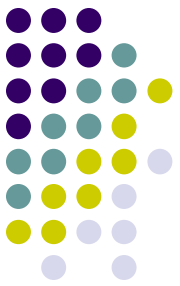


- Laundry Example
- Ann, Brian, Cathy, Dave each have one load of clothes to wash, dry, and fold
- Washer takes 30 minutes
- Dryer takes 40 minutes
- Folder takes 20 minutes

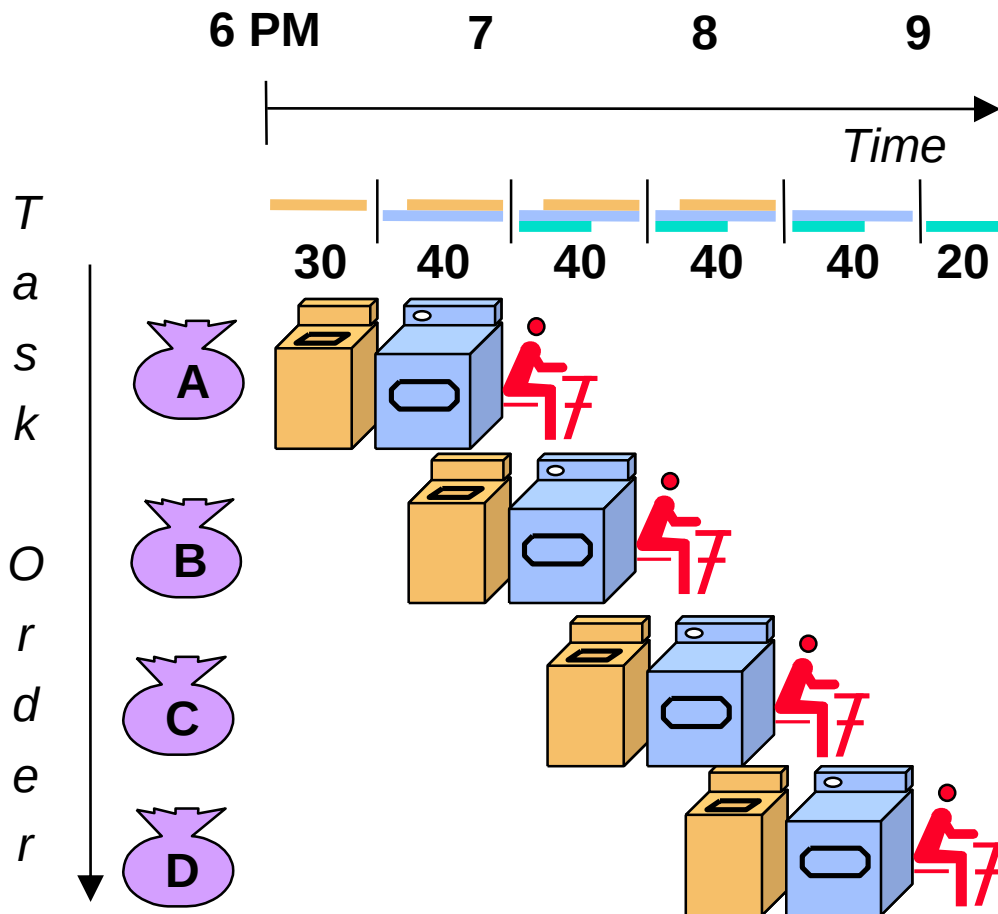






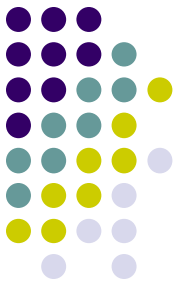


Traditional Pipeline Concept

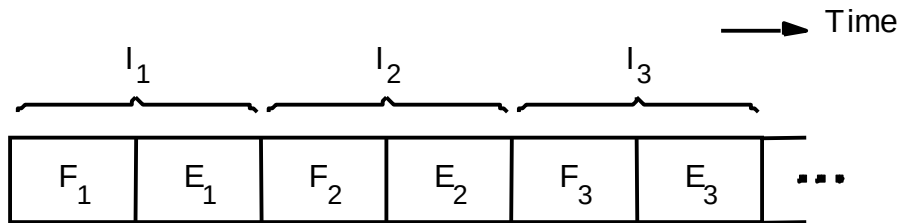


- Pipelining doesn't help latency of single task, it helps throughput of entire workload
- Pipeline rate limited by slowest pipeline stage
- Multiple tasks operating simultaneously using different resources
- Potential speedup = Number pipe stages
- Unbalanced lengths of pipe stages reduces speedup
- Time to “fill” pipeline and time to “drain” it reduces speedup
- Stall for Dependences

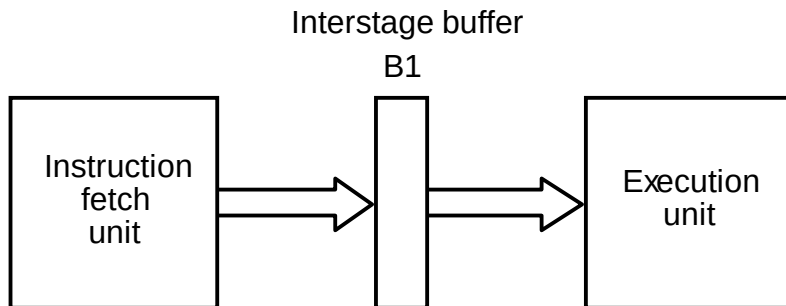
Use the Idea of Pipelining in a Computer



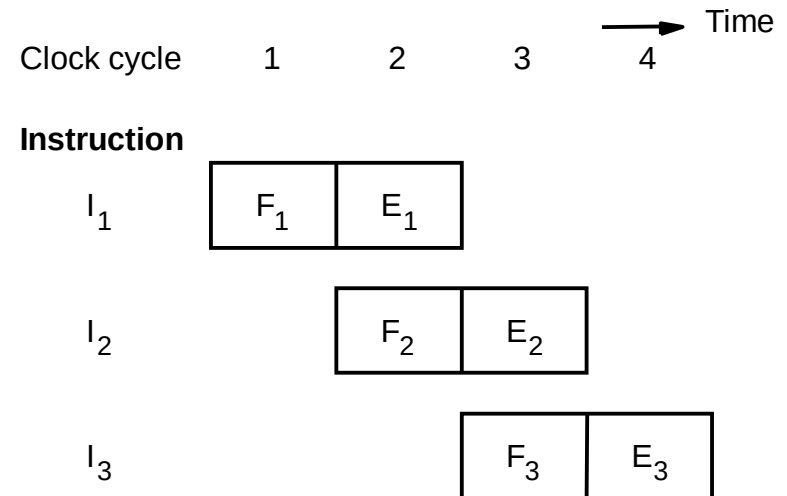
Fetch + Execution



(a) Sequential execution



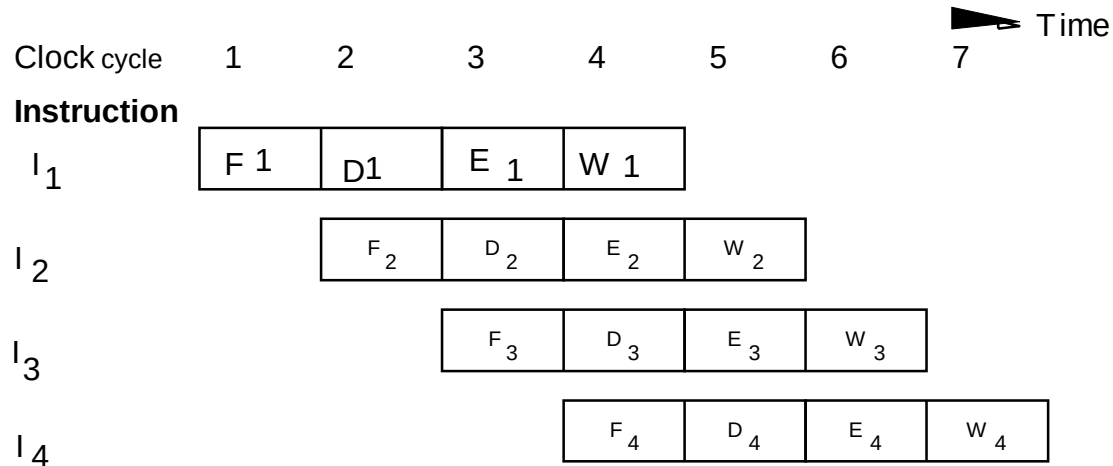
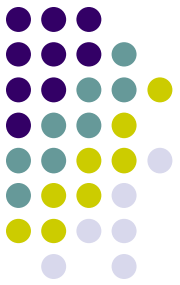
(b) Hardware organization



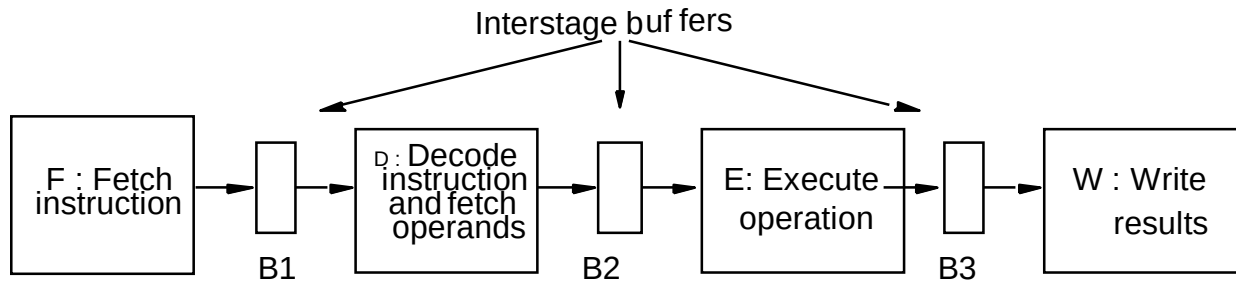
(c) Pipelined execution

Basic idea of instruction pipelining.

Fetch + Decode+ Execution + Write

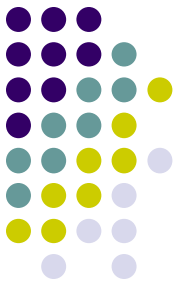


(a) Instruction execution divided into four steps



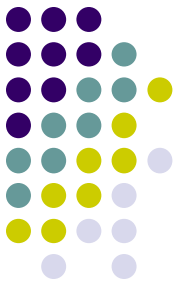
(b) Hardware organization

A 4-stage pipeline.



Role of Cache Memory

- Each pipeline stage is expected to complete in one clock cycle.
- The clock period should be long enough to let the slowest pipeline stage to complete.
- Faster stages can only wait for the slowest one to complete.
- Since main memory is very slow compared to the execution, if each instruction needs to be fetched from main memory, pipeline is almost useless.
- Fortunately, we have cache.



Pipeline Performance

- The potential increase in performance resulting from pipelining is proportional to the number of pipeline stages.
- However, this increase would be achieved only if all pipeline stages require the same time to complete, and there is no interruption throughout program execution.
- Unfortunately, this is not true.

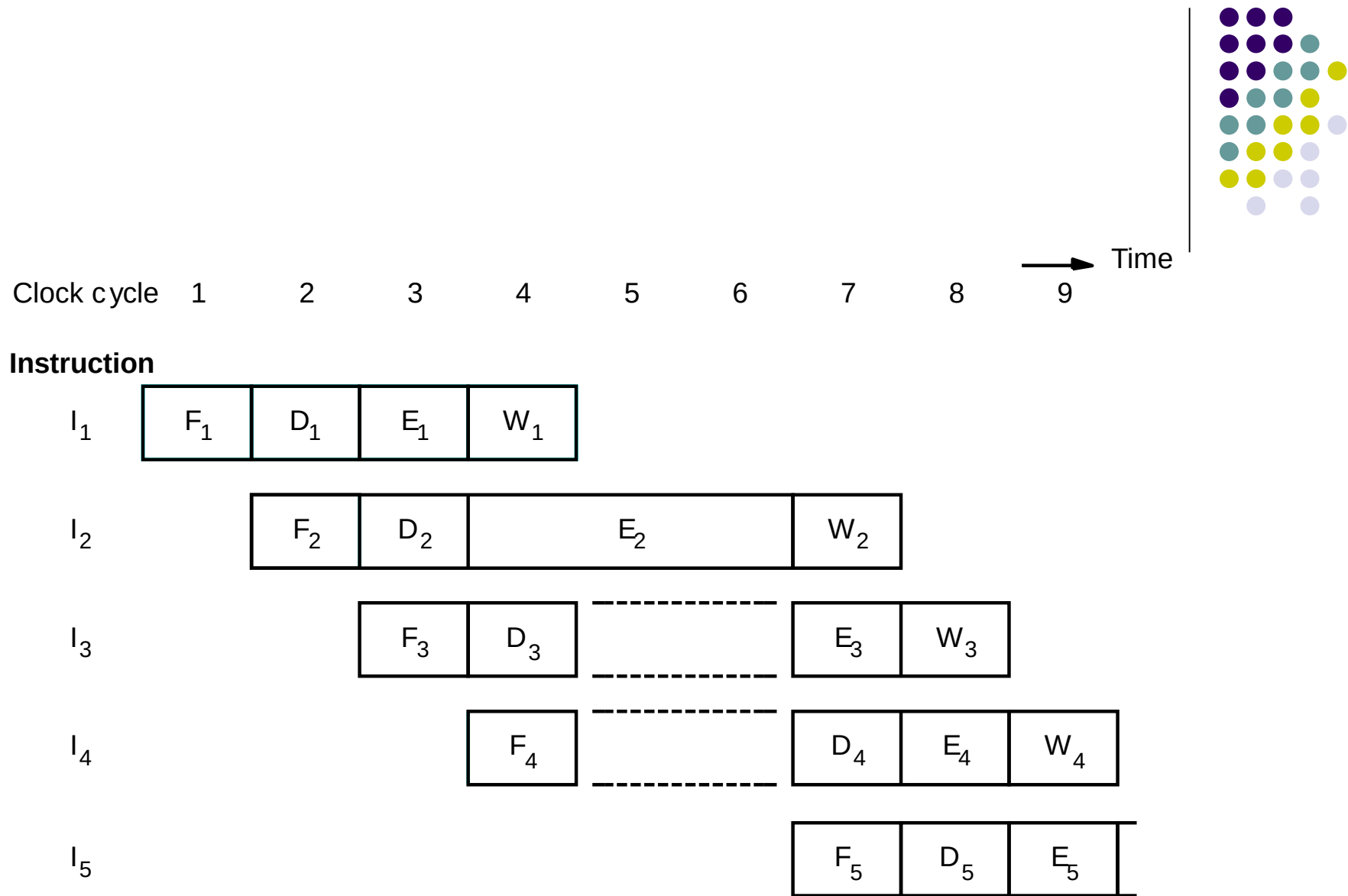


Figure: Effect of an execution operation taking more than one clock cycle.